

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/gdrive')

# Importation et nettoyage 'aide_alimentaire.csv'
aide = pd.read_csv('gdrive/My Drive/data/aide_alimentaire.csv', sep = ',')
aide
aide = aide.rename(columns={'Pays bénéficiaire': 'Pays', 'Année': 'Annee', 'Valeur': 'Quantite (tonne)'})
aide.info()
aide.Annee = aide.Annee.astype(object)
aide.Annee.unique()
aide.to_csv('gdrive/My Drive/data/aide_alimentaire_V2.csv', index=False)

# Importation et nettoyage 'dispo_alimentaire.csv'

dispo = pd.read_csv('gdrive/My Drive/data/dispo_alimentaire.csv')
dispo
dispo = dispo.rename(columns={'Zone': 'Pays'})
dispo = dispo[['Pays', 'Produit', 'Origine', 'Disponibilité alimentaire (Kcal/personne/jour)', 'Disponibilité alimentaire en quantité (kg/per
dispo = dispo.rename(columns={'Disponibilité intérieure': 'Disponibilité intérieure (en tonne)', 'Exportations - Quantité': 'Exportations -
dispo.dropna(how = 'all')
dispo = dispo.fillna(0)
dispo.sort_values(by=['Aliments pour animaux (en tonne)'], ascending=False).head(10)
dispo.to_csv('gdrive/My Drive/data/dispo_alimentaire_V2.csv', index=False)

# Importation et nettoyage 'population.csv'
pop = pd.read_csv('gdrive/My Drive/data/population.csv')
pop
pop.info()
pop = pop.rename(columns={'Zone': 'Pays', 'Année': 'Annee', 'Valeur': 'Population'})
pop.Population.unique()
pop.Annee.unique()
pop.Pays.unique()
pop.Population = pop.Population*1000
pop.Population = pop.Population.astype(int)
pop.Annee = pop.Annee.astype(object)
pop.info()
pop.to_csv('gdrive/My Drive/data/population_V2.csv', index=False)

# Importation et nettoyage 'sous_nutrition.csv'
nutri = pd.read_csv('gdrive/My Drive/data/sous_nutrition.csv')
nutri
nutri = nutri.rename(columns={'Zone': 'Pays', 'Année': 'Annee', 'Valeur': 'Population'})
nutri
nutri.Pays.unique()
nutri.Annee.unique()
nutri.Population.unique()
nutri = nutri.replace('<0.1', '0.05')
nutri = nutri.fillna(0)
nutri.Population.unique()
nutri['Annee'] = nutri['Annee'].map({'2012-2014': '2013', '2013-2015': '2014', '2014-2016': '2015', '2015-2017': '2016', '2016-2018': '2017', '2017
nutri.info()
nutri['Population'] = nutri['Population'].astype(float)
nutri.Population = nutri.Population*1000000
nutri['Population'] = nutri['Population'].astype(int)
nutri.info()
nutri.to_csv('gdrive/My Drive/data/sous_nutrition_V2.csv', index=False)

#Importation des fichiers nettoyer

nutri = pd.read_csv('gdrive/My Drive/data/sous_nutrition_V2.csv')
pop = pd.read_csv('gdrive/My Drive/data/population_V2.csv')
aide = pd.read_csv('gdrive/My Drive/data/aide_alimentaire_V2.csv')
dispo = pd.read_csv('gdrive/My Drive/data/dispo_alimentaire_V2.csv')

# 1 - La proportion de personnes en état de sous-nutrition :

# Jointure des DataFrame "nutri" et "pop" dans un seul DataFrame

dfproportion = pd.merge(nutri, pop, on=("Pays", "Annee"), how='inner')
dfproportion.head()

# Renommer colonnes pour différencier "Population Total" et la "Population en sous nutrition"

dfproportion = dfproportion.rename(columns={'Population_x': 'Pop_sous_nutrition', 'Population_y': 'Pop_total'})

```

```

# Filtrer les données sur l'année "2017"

pop17 = dfproportion.loc[dfproportion['Annee'] == 2017, :]
pop17.head()

# Calcul de la proportion de population en sous nutrition

cal = round((pop17['Pop_sous_nutrition']/pop17['Pop_total'])*100,2)

# Insertion de la colone "Proportion de sous nutrition (en %)" avec le resultat du calcul

pop17.insert(4,"Proportion de sous nutrition (en %)",cal,allow_duplicates=False)

# Population total mondial en 2017

total_pop = pop17['Pop_total'].sum()

# Population total mondial en sous nutrition en 2017

total_pop_sous_nutrition = pop17['Pop_sous_nutrition'].sum()

# Proportion sous nutrition mondial

cal_proportion_mondial = round((total_pop_sous_nutrition/total_pop)*100,2)

print("Nombre de personnes en état de sous-nutrition : ", total_pop_sous_nutrition)
print("Proportion de la population mondiale en sous nutrition : ", cal_proportion_mondial, "%")

#2- Le nombre théorique de personnes qui pourraient être nourries :

# Création d'un DataFrame pour la disponibilité alimentaire

dispo_alim = dispo[['Pays', 'Disponibilité alimentaire (Kcal/personne/jour)']]
dispo_alim = dispo_alim.groupby('Pays').sum()
dispo_alim.head()

# Merge entre la disponibilité alimentaire et la population en 2017

pop_dispo = pd.merge(dispo_alim, pop17, on=("Pays"), how='inner')
pop_dispo = pop_dispo[['Pays', 'Disponibilité alimentaire (Kcal/personne/jour)', 'Pop_total']]
pop_dispo.head()

# Calcul du nombre de kcal par jour et par personne dispo , multiplier par le nombre de personne

kcalDispo = round(((pop_dispo["Disponibilité alimentaire (Kcal/personne/jour)"] * pop_dispo["Pop_total"])).sum())

# 2500 : nombre de calorie moyen pour un adulte
nb_pers_nourri = round(kcalDispo/2500)

cal_proportion_mondial_theorie = round((nb_pers_nourri/total_pop)*100,2)

print("Le nombre théorique de personnes qui pourraient être nourries est de",nb_pers_nourri,"personnes.""Soit",cal_proportion_mondial_1

#3- Le nombre théorique de personnes qui pourraient être nourries avec disponibilité alimentaire des produits végétaux :

# Création d'un DataFrame pour la disponibilité alimentaire des produits végétaux

dispo_vege = dispo[['Pays', 'Origine', 'Disponibilité alimentaire (Kcal/personne/jour)']]
dispo_veg = dispo_vege['Origine'] == 'vegetale'
dispo_vegetal = dispo_vege[dispo_veg].groupby('Pays').sum()
dispo_vegetal.head()

# Merge entre la disponibilité alimentaire des produits végétaux et la population en 2017

pop_dispo_veg = pd.merge(dispo_vegetal, pop17, on=("Pays"), how='inner')
pop_dispo_veg = pop_dispo_veg[['Pays', 'Disponibilité alimentaire (Kcal/personne/jour)', 'Pop_total']]
pop_dispo_veg = pop_dispo_veg.rename(columns={'Disponibilité alimentaire (Kcal/personne/jour)':'Disponibilité alimentaire vegetale (Kcal/personne/jour)'})
pop_dispo_veg.head()

# Calcul du nombre de kcal par jour et par personne dispo , multiplier par le nombre de personne

kcalDispo_veg = round(((pop_dispo_veg["Disponibilité alimentaire vegetale (Kcal/personne/jour)"] * pop_dispo_veg["Pop_total"])).sum())

# 2500 : nombre de calorie moyen pour un adulte
nb_pers_nourri_veg = round(kcalDispo_veg/2500)

```

```

cal_proportion_mondial_veg = round((nb_pers_nourri_veg/total_pop)*100,2)

print("Le nombre théorique de personnes qui pourraient etre nourries, seulement grace à la disponibilité alimentaire des produits végétaux est de : ", cal_proportion_mondial_veg)

# 4 - Répartition de la disponibilité intérieure

# Création d'un DataFrame avec seulement les données importantes
proportion_dispo = dispo[['Pays', 'Produit', 'Origine', 'Aliments pour animaux (en tonne)', 'Pertes (en tonne)', 'Nourriture (en tonne)']]

# Calcul de la disponibilité totale
cal_dispo_total = dispo['Importations - Quantité (en tonne)'] + dispo['Production (en tonne)']

# Insertion de la colonne "Dispo Total (en tonne)" avec le résultat du calcul
proportion_dispo.insert(6, "Dispo Total (en tonne)", cal_dispo_total, allow_duplicates=False)

# Regroupement des données par pays et somme des colonnes par pays
proportion_dispo = proportion_dispo.groupby('Pays').sum()

# Calcul des proportions
cal_proportion_dispo_animaux = round((proportion_dispo['Aliments pour animaux (en tonne)'] / proportion_dispo['Dispo Total (en tonne)']) * 100, 2)
cal_proportion_dispo_pertes = round((proportion_dispo['Pertes (en tonne)'] / proportion_dispo['Dispo Total (en tonne)']) * 100, 2)
cal_proportion_dispo_nourriture = round((proportion_dispo['Nourriture (en tonne)'] / proportion_dispo['Dispo Total (en tonne)']) * 100, 2)

# Insertion des colonnes de proportions
proportion_dispo.insert(4, "Proportion des aliments pour animaux par rapport à la disponibilité (en %)", cal_proportion_dispo_animaux, allow_duplicates=True)
proportion_dispo.insert(5, "Proportion des pertes par rapport à la disponibilité (en %)", cal_proportion_dispo_pertes, allow_duplicates=True)
proportion_dispo.insert(6, "Proportion de la nourriture par rapport à la disponibilité (en %)", cal_proportion_dispo_nourriture, allow_duplicates=True)

# Proportions mondiales
prop_mondial_animaux = round((proportion_dispo['Aliments pour animaux (en tonne)'].sum() / proportion_dispo['Dispo Total (en tonne)'].sum()) * 100, 2)
prop_mondial_pertes = round((proportion_dispo['Pertes (en tonne)'].sum() / proportion_dispo['Dispo Total (en tonne)'].sum()) * 100, 2)
prop_mondial_nourriture = round((proportion_dispo['Nourriture (en tonne)'].sum() / proportion_dispo['Dispo Total (en tonne)'].sum()) * 100, 2)

# Affichage des résultats
print("Proportion mondial des aliments pour animaux par rapport à la disponibilité : ", prop_mondial_animaux, "%")
print("Proportion mondial des pertes par rapport à la disponibilité : ", prop_mondial_pertes, "%")
print("Proportion mondial de la nourriture par rapport à la disponibilité : ", prop_mondial_nourriture, "%")

# Création d'un graphique à barres pour visualiser les proportions
labels = ['Aliments pour animaux', 'Pertes', 'Nourriture']
sizes = [prop_mondial_animaux, prop_mondial_pertes, prop_mondial_nourriture]

plt.figure(figsize=(8, 6))
bars = plt.bar(labels, sizes, color=['#ff9999', '#66b3ff', '#99ff99'])

# Ajouter les valeurs au-dessus des barres
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, f'{yval}%', ha='center', va='bottom')

plt.ylabel('Proportion (%)')
plt.ylim(0, 100) # Limiter l'axe Y à 100%
plt.grid(axis='y')

plt.show()

# 5 - Part de l'utilisation des principales céréales entre l'alimentation humaine et animale
cereales = ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle', 'Avoine', 'Millet', 'Sorgho', 'Céréales', 'Autres']

# Filtrer les données pour les céréales
dispo_cereale = dispo[dispo['Produit'].isin(cereales)]

# Calculer la somme de l'utilisation pour l'alimentation humaine et animale
alimentation_humaine = dispo_cereale.groupby('Produit')['Nourriture (en tonne)'].sum()
alimentation_animale = dispo_cereale.groupby('Produit')['Aliments pour animaux (en tonne)'].sum()

# Créer un DataFrame avec les résultats
resultats_cereales = pd.DataFrame({
    'Céréale': alimentation_humaine.index,
    'Alimentation Humaine (tonnes)': alimentation_humaine.values,
    'Alimentation Animale (tonnes)': alimentation_animale.values
}).reset_index(drop=True)

# Calculer la part d'utilisation pour chaque type
resultats_cereales['Part Alimentation Humaine (%)'] = round((resultats_cereales['Alimentation Humaine (tonnes)'] /
    (resultats_cereales['Alimentation Humaine (tonnes)'] +
    resultats_cereales['Alimentation Animale (tonnes)'])) * 100, 2)

```

```

resultats_cereales['Part Alimentation Animale (%)'] = round((resultats_cereales['Alimentation Animale (tonnes)'] /
                                                             (resultats_cereales['Alimentation Humaine (tonnes)'] +
                                                              resultats_cereales['Alimentation Animale (tonnes)'])) * 100, 2)

# Réorganiser les colonnes pour une meilleure présentation
resultats_cereales = resultats_cereales[['Céréale', 'Alimentation Humaine (tonnes)', 'Part Alimentation Humaine (%)',
                                         'Alimentation Animale (tonnes)', 'Part Alimentation Animale (%)']]

# Afficher le tableau final avec un meilleur formatage
print(resultats_cereales.to_string(index=False))

# 6 - Les pays pour lesquels la proportion de personnes sous-alimentées est la plus forte en 2017
# Reprise du DataFrame avec classement dans l'ordre décroissant sur la colonne "Proportion de sous nutrition (en %)"
top_sous_nutrition = pop17.sort_values(by=['Proportion de sous nutrition (en %)', ascending=False]).head(10)

# Affichage des résultats
print(top_sous_nutrition)

# Création d'un graphique à barres verticales pour visualiser les 10 pays ayant la plus forte proportion de sous-alimentation
plt.figure(figsize=(12, 8))
bars = plt.bar(top_sous_nutrition['Pays'], top_sous_nutrition['Proportion de sous nutrition (en %)', color='#FF9800', edgecolor='black

# Ajouter des étiquettes de données sur les barres
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, f'{yval:.1f}%', ha='center', va='bottom', fontsize=10)

# Améliorations esthétiques
plt.xlabel('Pays', fontsize=14)
plt.ylabel('Proportion de Sous-Nutrition (%)', fontsize=14)
plt.title('Top 10 des Pays avec la Plus Forte Proportion de Sous-Alimentation en 2017', fontsize=16)
plt.xticks(rotation=45, ha='right') # Rotation des étiquettes de l'axe X
plt.ylim(0, top_sous_nutrition['Proportion de sous nutrition (en %)'].max() + 5) # Ajuster l'axe Y pour un meilleur espacement
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Afficher le graphique
plt.tight_layout() # Ajuste le layout pour éviter le chevauchement
plt.show()

# 7 - Liste des 10 pays qui ont le plus bénéficié de l'aide alimentaire entre 2013 et 2016
# Filtrer les données pour les années 2013 à 2016
aide_filtered = aide[(aide['Annee'] >= 2013) & (aide['Annee'] <= 2016)]

# Regroupement des données d'aide alimentaire par "Pays" et calcul de la somme de l'aide par pays
aidealim = aide_filtered.groupby('Pays')['Quantite (tonne)'].sum().reset_index()

# Renommer la colonne pour plus de clarté
aidealim = aidealim.rename(columns={'Quantite (tonne)': 'Total_Aide'})

# Affichage dans l'ordre décroissant des 10 pays qui ont reçu le plus d'aide
top_aidealim = aidealim.sort_values(by='Total_Aide', ascending=False).head(10)

# Affichage final
print(top_aidealim)

# Création d'un graphique à barres verticales pour visualiser les 10 pays ayant reçu le plus d'aide
plt.figure(figsize=(12, 8))
bars = plt.bar(top_aidealim['Pays'], top_aidealim['Total_Aide'], color='#4CAF50', edgecolor='black')

# Ajouter des étiquettes de données sur les barres
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, f'{yval:.0f}', ha='center', va='bottom', fontsize=10)

plt.xlabel('Pays', fontsize=14)
plt.ylabel('Total d\'Aide Alimentaire (tonnes)', fontsize=14)
plt.title('Top 10 des Pays ayant Bénéficié de l\'Aide Alimentaire (2013-2016)', fontsize=16)
plt.xticks(rotation=45, ha='right') # Rotation des étiquettes de l'axe X
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Afficher le graphique
plt.tight_layout() # Ajuste le layout pour éviter le chevauchement
plt.show()

# 8 - Évolution de l'aide alimentaire pour les 5 pays qui en ont le plus bénéficié entre 2013 et 2016

```

```
# Filtrer les données pour les années 2013 à 2016
aide_filtered = aide[aide['Annee'] >= 2013] & (aide['Annee'] <= 2016)]

# Regroupement des données d'aide alimentaire par "Pays" et "Année" et calcul de la somme de l'aide
aide_evolution = aide_filtered.groupby(['Pays', 'Annee'])['Quantite (tonne)'].sum().reset_index()

# Regroupement par pays pour calculer la somme totale de l'aide
total_aide = aide_evolution.groupby('Pays')['Quantite (tonne)'].sum().reset_index()

# Identification des 5 pays ayant reçu le plus d'aide
top_5_pays = total_aide.sort_values(by='Quantite (tonne)', ascending=False).head(5)

# Filtrer les données d'évolution pour ces 5 pays
evolution_top_5 = aide_evolution[aide_evolution['Pays'].isin(top_5_pays['Pays'])]

# Pivot pour obtenir un tableau avec les pays en lignes et les années en colonnes
tableau_final = evolution_top_5.pivot(index='Pays', columns='Annee', values='Quantite (tonne)').fillna(0)

# Affichage du tableau final
print(tableau_final)

# Création du graphique
plt.figure(figsize=(10, 6))
for pays in top_5_pays['Pays']:
    subset = evolution_top_5[evolution_top_5['Pays'] == pays]
    plt.plot(subset['Annee'], subset['Quantite (tonne)'], marker='o', label=pays)

# Ajout des détails au graphique
plt.title('Évolution de l\'aide alimentaire (2013-2016)')
plt.xlabel('Annee')
plt.ylabel('Quantité d\'aide (tonnes)')
plt.xticks(subset['Annee'].unique()) # Afficher toutes les années
plt.legend(title='Pays')
plt.grid()
plt.tight_layout()

# Affichage du graphique
plt.show()

# 9 - Liste des 10 pays ayant la plus forte disponibilité alimentaire par habitant
# Regroupement par "Pays" et calcul de la somme de la "Disponibilité alimentaire (Kcal/personne/jour)" par pays
dispo = dispo.groupby('Pays')['Disponibilité alimentaire (Kcal/personne/jour)'].sum()

# Passage en DataFrame
DFdispo = pd.DataFrame(dispo).reset_index()

# Conversion de float vers int
DFdispo['Disponibilité alimentaire (Kcal/personne/jour)'] = DFdispo['Disponibilité alimentaire (Kcal/personne/jour)'].astype(int)

# Affichage des disponibilités alimentaires par pays
top_dispo = DFdispo.sort_values(by='Disponibilité alimentaire (Kcal/personne/jour)', ascending=False).head(10)
print(top_dispo)

# Création d'un graphique à barres verticales pour visualiser les 10 pays ayant la plus forte disponibilité alimentaire
plt.figure(figsize=(12, 8))
bars = plt.bar(top_dispo['Pays'], top_dispo['Disponibilité alimentaire (Kcal/personne/jour)'], color='#2196F3', edgecolor='black')

# Ajouter des étiquettes de données sur les barres
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, f'{yval}', ha='center', va='bottom', fontsize=10)

# Améliorations esthétiques
plt.xlabel('Pays', fontsize=14)
plt.ylabel('Disponibilité Alimentaire (Kcal/personne/jour)', fontsize=14)
plt.title('Top 10 des Pays ayant la Plus Forte Disponibilité Alimentaire par Habitant', fontsize=16)
plt.xticks(rotation=45, ha='right') # Rotation des étiquettes de l'axe X
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Afficher le graphique
plt.tight_layout() # Ajuste le layout pour éviter le chevauchement
plt.show()

# 10 - Liste des 10 pays ayant la plus faible disponibilité alimentaire par habitant
# Regroupement par "Pays" et calcul de la somme de la "Disponibilité alimentaire (Kcal/personne/jour)" par pays
dispo = dispo.groupby('Pays')['Disponibilité alimentaire (Kcal/personne/jour)'].sum()

# Passage en DataFrame
DFdispo = pd.DataFrame(dispo).reset_index()
```

```

# Conversion de float vers int
DFdispo['Disponibilité alimentaire (Kcal/personne/jour)'] = DFdispo['Disponibilité alimentaire (Kcal/personne/jour)'].astype(int)

# Affichage des disponibilités alimentaires par pays
bottom_dispo = DFdispo.sort_values(by=['Disponibilité alimentaire (Kcal/personne/jour)'], ascending=True).head(10)
print(bottom_dispo)

# Création d'un graphique à barres verticales pour visualiser les 10 pays ayant la plus faible disponibilité alimentaire
plt.figure(figsize=(12, 8))
bars = plt.bar(bottom_dispo['Pays'], bottom_dispo['Disponibilité alimentaire (Kcal/personne/jour)'], color='#F44336', edgecolor='black')

# Ajouter des étiquettes de données sur les barres
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, f'{yval}', ha='center', va='bottom', fontsize=10)

# Améliorations esthétiques
plt.xlabel('Pays', fontsize=14)
plt.ylabel('Disponibilité Alimentaire (Kcal/personne/jour)', fontsize=14)
plt.title('Top 10 des Pays ayant la Plus Faible Disponibilité Alimentaire par Habitant', fontsize=16)
plt.xticks(rotation=45, ha='right') # Rotation des étiquettes de l'axe X
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Afficher le graphique
plt.tight_layout() # Ajuste le layout pour éviter le chevauchement
plt.show()

# 1 - Proportion de personnes en état de sous-nutrition en Thaïlande pour 2018
df_thaïlande = pd.merge(nutri[nutri['Pays'] == 'Thaïlande'],
                        pop[pop['Pays'] == 'Thaïlande'],
                        on="Annee",
                        how='inner')

# Renommer les colonnes pour différencier "Population Total" et "Population en sous nutrition"
df_thaïlande = df_thaïlande.rename(columns={'Population_x': 'Pop_sous_nutrition',
                                           'Population_y': 'Pop_total'})

# Filtrer pour l'année 2018
df_thaïlande_2018 = df_thaïlande[df_thaïlande['Annee'] == 2018]

# Calcul de la proportion de population en sous nutrition
df_thaïlande_2018['Proportion de sous nutrition (en %)'] = round((df_thaïlande_2018['Pop_sous_nutrition'] / df_thaïlande_2018['Pop_total']) * 100, 2)

# Affichage des résultats pour 2018
print(df_thaïlande_2018[['Annee', 'Proportion de sous nutrition (en %)']])

# Production et exportation de Manioc en Thaïlande :

# Filtrer les données pour la Thaïlande et le manioc
manioc_thaïlande = dispo[(dispo['Pays'] == 'Thaïlande') &
                        (dispo['Produit'] == 'Manioc')]

# Récupérer la production et les exportations
production_manioc = manioc_thaïlande['Production (en tonne)'].sum()
exportations_manioc = manioc_thaïlande['Exportations - Quantité (en tonne)'].sum()

# Affichage des résultats
print(f"Production totale de manioc en Thaïlande : {production_manioc:.2f} tonnes")
print(f"Exportations totales de manioc en Thaïlande : {exportations_manioc:.2f} tonnes")

# Filtrer les données pour la Thaïlande
dispo_thaïlande = dispo[dispo['Pays'] == 'Thaïlande']

# Calculer la disponibilité alimentaire totale (en tonnes)
disponibilite_totale = dispo_thaïlande['Disponibilité intérieure (en tonne)'].sum()

# Supposons que vous avez un DataFrame 'pop' avec la population de la Thaïlande
population_thaïlande = pop[pop['Pays'] == 'Thaïlande']['Population'].sum()

# Calculer la disponibilité alimentaire par habitant (en kg)
disponibilite_par_habitant = (disponibilite_totale * 1000) / population_thaïlande # Convertir en kg

# Affichage du résultat
print(f"Disponibilité alimentaire par habitant en Thaïlande : {disponibilite_par_habitant:.2f} kg par habitant")

```

